MODULE 7.4

Cloud Application Security

Secure Coding | Input Validation | Integrity Checks

Presented by: Kudzaishe Majeza

Cisco Networking Academy | 2026

Topics:

- 7.4.1 Application Development
- 7.4.2 Secure Coding Techniques
- 7.4.3 Input Validation
- 7.4.4 Validation Rules
- 7.4.5 Integrity Checks
- 7.4.6 Other Application Security Practices
- 7.4.7 Check Your Understanding — Application Security
- Summary
- References

Application Development

- **Most real breaches start in the code, not the network:** attackers increasingly target the application itself, because a flaw in the code reaches straight past network-layer defenses like firewalls and ACLs.
- **The SDLC:** every application moves through a Software Development Life Cycle — Requirements, Design, Implementation, Verification, and Release & Maintenance — and security can be built in, or skipped, at each stage.
- **"Shift left":** traditionally, security testing happened right before release. Shifting left means building security requirements and testing in from the very first stage instead, because a flaw caught during design costs far less to fix than the same flaw caught in production.

SDLC Stage	Security Activity
Requirements	Define security requirements alongside functional ones (e.g., "passwords must be hashed").
Design	Threat modeling — identify what could go wrong before a line of code is written.
Implementation	Secure coding practices and peer code review.
Verification	Static and dynamic testing, penetration testing.
Release & Maintenance	Patch management and ongoing vulnerability monitoring.

Secure Coding Techniques

Principle	What It Means
Validate all input	Never trust data from a user, file, or another system until it's been checked.
Minimize attack surface	Expose only the functionality that's actually needed — every extra feature is an extra risk.
Fail securely	When something goes wrong, default to denying access, not granting it.
Defense in depth	Layer multiple controls so one failure doesn't mean a full breach.
Least privilege	Code — and the account it runs under — should have the minimum access needed to function.
Don't trust the client	Client-side checks (like JavaScript validation) improve the user experience but can be bypassed — always re-check on the server.

Input Validation

- **What it is:** checking that data entered by a user, or received from another system, matches the expected type, format, length, and range before the application acts on it.
- **Where it must happen:** client-side validation improves the user experience, but it is trivially bypassed by an attacker calling the underlying API directly — the server must independently validate everything, every time.
- **Allow-list vs. deny-list:** an allow-list (whitelist) defines exactly what's permitted and rejects everything else; a deny-list (blacklist) tries to block known-bad patterns. Allow-lists are strongly preferred, because a deny-list only stops attacks someone already thought of.
- **What it prevents:** input validation is the primary defense against injection attacks — SQL injection, cross-site scripting (XSS), and command injection all rely on unvalidated input reaching a sensitive function.

Validation Rules

Input validation is enforced through a combination of concrete rules, applied together:

Rule Type	Example
Data type	A "quantity" field must be an integer, not text.
Format / pattern	An email field must match name@domain.tld.
Length	A username must be between 3 and 20 characters.
Range	An "age" field must be between 0 and 120.
Presence	A required field cannot be submitted empty.
Allow-list of values	A "country" field must match an entry in a known list.

Integrity Checks

- **A different problem than validation:** input validation catches bad data on the way in — integrity checks confirm data hasn't been altered after it was created or while it was in transit.
- **Hashing:** running data through a one-way function like SHA-256 produces a fixed-length fingerprint; even a one-character change produces a completely different hash, making tampering obvious.
- **HMAC (Hash-based Message Authentication Code):** combines a hash function with a secret key, so the receiver can confirm both that the data wasn't altered and that it came from someone who knows the shared secret.
- **Digital signatures:** use public-key cryptography so a recipient can verify both integrity and the sender's identity, without ever needing to share a secret key in advance.

Abbreviations: SHA = Secure Hash Algorithm. HMAC = Hash-based Message Authentication Code.

Other Application Security Practices

- **Code review & static analysis (SAST):** scanning source code for known-insecure patterns before it's ever compiled or deployed.
- **Dynamic testing (DAST):** testing the running application from the outside, the way an attacker would, to catch issues static analysis alone can't see.
- **Secure session management:** session tokens must be long, random, transmitted only over HTTPS, and invalidated on logout — a predictable or leaked session token is as good as a stolen password.
- **Proper error handling:** error messages shown to users should never include stack traces, database queries, or internal file paths — that information hands an attacker a map of the application.
- **Dependency scanning:** most applications are built on dozens of third-party libraries; a single outdated, vulnerable dependency can compromise the whole application even if the custom code is flawless.

Abbreviations: SAST = Static Application Security Testing. DAST = Dynamic Application Security Testing.

Check Your Understanding

Read the scenario, then identify the concept it illustrates:

Q: A form only checks input in JavaScript in the browser, and an attacker bypasses it by calling the API directly.

A: Missing server-side validation

Q: A field rejects anything that isn't one of five pre-approved country codes.

A: Allow-list (whitelist) validation

Q: A downloaded file's SHA-256 hash is compared against the one published by the vendor.

A: An integrity check

Q: An error page shows the full database query and a file path after a failed login.

A: Improper error handling — leaking information an attacker can use

Q: A security requirement ("passwords must be hashed") is written before any code exists.

A: Shift left — security built in at the Requirements stage

Q: An application is found to be vulnerable only because of an outdated third-party library.

A: A dependency vulnerability

Summary

- **7.4.1–7.4.2 —** security belongs in every stage of the SDLC, not just testing right before release, and secure coding comes down to a handful of repeatable principles: validate input, minimize attack surface, fail securely, and never trust the client.
- **7.4.3–7.4.4 —** input validation — enforced server-side with allow-lists and concrete rules for type, format, length, and range — is the primary defense against injection attacks.
- **7.4.5 —** integrity checks (hashing, HMAC, digital signatures) solve a different problem than validation: confirming data wasn't altered after it was created, not just that it looks well-formed.
- **7.4.6 —** code review, dynamic testing, secure session management, careful error handling, and dependency scanning round out application security beyond the code an organization actually writes itself.

References

Cisco Networking Academy — Network Defense, Module 7.4: Cloud Application Security

https://www.netacad.com

OWASP — Secure Coding Practices Quick Reference Guide

https://owasp.org/www-project-secure-coding-practices-quick-reference-guide/

NIST — Secure Software Development Framework (SP 800-218)

https://csrc.nist.gov/publications/detail/sp/800-218/final